Parametrized and Exact Algorithms

Notes by Arjun Maneesh Agarwal

based on course by G. Philip

The main book will be Parametrized Algorithms by Cygan et. al (chapters 1 to 7, 13 and 15). Other than that < copy from moodle >.

Table of Contents

1. Philip's Wisdom 2

2. Brief Intro and Motivation 2

3. Definitions 4

4. Exact Exponential Algorithms 5

5. Kernalization 5

5.1. Planar Independent Set 7

6. Feedback Vertex Set 8

7. Feedback Arc Set 8

8. Feedback Vertex Set Revisited 10

9. Crown Decomposition 11

10. An $\mathcal{O}^*(2k)$ Kernel for Vertex Cover 13

11. Sunflower Lemma 14

12. Iterated Compression 15

13. Tournament Feedback Vertex Set 15

14. Randomized Algorithms 16

14.1. Color Coding 16

14.2. Random Separation 18

14.3. Divide and Color 21

14.4. Chromatic Coding 23

15. Graph Minors 25

16. I was home for like 2-3 classes 28

17. DP over Subsets 28

17.1. Set Cover 28

17.2. Steiner Tree 29

17.2.1. An *Avent la lettre* FPT Algorithms 30

18. Fast Subset Convalution 31

18.1. Chormatic Number 33

18.2. Dreyphus-Wagner 33

19. Back to Steiner Tree! 34
19.1. Unweighted Steiner Tree 34
20. Tree Width 37

1. Philip's Wisdom

"Why do you want to solve an algorithms problem, probably to make more money. Or kill more people. Or maybe save more people."

"They are a complexity theorist. They want nothing to do with algorithms."

"Sometimes, nobody knows the algorithm. Not even LLMs."

"Tar(z|j)an jumped in to solve it."

"We will now talk about minors. Not legal minors, graph minors."

"I don't know how to pronounce this. For that you'll have to ask the French prof. They'll only pronounce 3 letters. I know that only 10% of the letters will be pronounced, I just don't know which 10%."

"Now that I have done French, I will do German."

"See this where my knowledge of German helps, to waste time."

(about a benchmark algorithm) "He was just starting PhD when he found it. So it was not in final year of masters. No need to worry."

"I like saying Principal of Inclusion-Exclusion is a special case of Mobius inversion because it sounds smart. I don't know how to prove it. I might have held it my head at some point, but maybe I should now ask LLM to explain it to me like I am five."

2. Brief Intro and Motivation

Parametrized Algorithms starts where the first algorithms course ends: NP Completeness.

Most algorithms we saw in the algorithms course have complexity $O(n^c)$ for some constant c. This raises the question if that is true of all problems, sadly, this is where NP Hardness and Completeness comes in. While we don't know if we can solve them polytime, these NP Complete problems span thousands of fields and experts in their field have tried finding fast algorithms. As these problems are complete, all reduce to each other which implies that if one of them succeed, all of them succeed. Unfortunately, they have all failed, which is the sort of silence that speaks quite loudly.

So how do we cope with this? One is Approx Algorithms. The other idea is Random Algorithms. Another is heuristics and meta-heuristics (taken from physics and biology and what not).

In theory, NP Completeness implies all the problems are equally hard modulo some polynomial time translations. However, some problems were easier to solve in practice.

Example

Type Inference for Hindley-Milner type system is unconditionally not solvable in polynomial time (in the size n of the input programme) as it hard for the class EXP (that is problems that can be solved in $O(2^{\text{poly}})$ time) (from Maivson, POPL 1990) and P $\subsetneq$ EXP (Time Hierarchy theorem)

However, we don't think about the size of programme while writing code. And the compiler happily (or sadly) infers the type quite quickly.

Theorem 2.1. *Type inference for Hindley-Milner Type ststem can be solved in $O(2^k n)$ time where*

- *n is the size of input programme,*
- *k is the nesting depth of types in the programmes*

(Lichtenstein and Porveli, POPL 1985)

The idea is to pick something that tends to something that is usually small in practice and limit the exponential blow up to that parameter.

So our roadmap is: pick a parameter k that could be natural or something more convoluted, try to find an algorithm of time complexity $O(f(k)n^c)$. If we succeed the problem is Fixed Parameter Tractable aka FPT.

Definition: Vertex Cover

Vertex cover of a graph G is a subset $S \subseteq V(G)$ such that $G \setminus S$ has no edges.

Definition: Vertex Cover (Standard Parametrization)

Input: Graph G, $k \in \mathbb{N}$

Parameter: k

Question: Does G have a vertex cover of size $\leq k$?

Remark

This can be thought of as choosing a subset as guard posts such that all the roads are guarded.

Also, please make sure to not confuse this with **Dominating Set** (which is choosing vertices to put 1 edge radius umbrellas to cover everyone).

While vertex cover is NP Complete, we can see a simple algorithm that runs in $\mathcal{O}^*(2^k)$ time (we can also do it in $\mathcal{O}^*(1.4656^k)$ time as we'll later see and in $\mathcal{O}^*(1.2529^k)$ as we shall not see).

The idea is that given an edge, one of the two vertices neighboring it have to be in the vertex cover. This is a local property and can be abused to get the algorithm.

Vertex Cover In $O(2^k n)$ time

```
1 func existVertexCover(G, k):
2     pick edge uv ∈ V(G)
3     return existVertexCover( G \ {u}, k − 1) OR existVertexCover( G \ {v}, k − 1)
```

3. Definitions

Definition: Classical Language

Let Σ be a finite alphabet, for example $\Sigma = \{0, 1\}$ or $\Sigma =$ ASCII. A classical language over Σ is $L \subseteq \Sigma^*$, for example $L = \{$All valid Haskell programmes$\}$.

Definition: Parametrized Language

A parametrized language over Σ is $L \subseteq \Sigma^* \times \mathbb{N}$, for example $L = \{(x, k) \mid x$ is a valid

For a fixed k, $L_k = \{(x, k) \mid (x, k) \in L\}$ is the k-th slice of L

Definition: Slice-wise Polynomial Time (XP)

A parametrized language L is slice-wise polynomial time solvable (XP) if there exists an algorithm $\mathcal{A}$ and a computable function f such that:

- For all x, k, $\mathcal{A}$ runs in $\leq |x|^{f(k)}$ time on input (x, k)
- $(x, k) \in L \iff \mathcal{A}((x, k)) =$ **YES**

Definition: Fixed Parameter Tractable (FPT)

A parametrized language L is fixed parameter tractable (FPT) if there exists an algorithm $\mathcal{A}$ and a computable function f such that:

- For all x, k, $\mathcal{A}$ runs in $\leq f(k)\ |x|^c$ time on input (x, k) where c is a constant independent of $|x|$ and k.
- $(x, k) \in L \iff \mathcal{A}((n, k)) =$ **YES**.

For notational convenience, $f(k)\ |x|^c$ is abbreviated $\mathcal{O}^*(f(k))$.

One motivation could be this table:

$\frac{n^{k+1}}{2^k n}$	$n = 50$	$n = 100$	$n = 150$
$k = 2$	625	2500	5625
$k = 3$	15625	125000	421875

$k = 10$	10^{12}	8×10^{13}	3.7×10^{16}
$k = 20$	1.8×10^{26}	5×10^{30}	2.1×10^{35}

4. Exact Exponential Algorithms

An example could be Hamiltonian Path.

Definition: Hamiltonian Path

Input: Given a graph G and vertex $v \in V$

Question: Does there exist a path P starting at v which covers all vertices of G without repeating vertices or edges.

This problem is NP Hard, however using a DP on sets, we can do this in $\mathcal{O}^*(2^n)$ instead of the brute force $\mathcal{O}^*(n!)$.

Another example is Chromatic Number

Definition: Chromatic Number

Input: Given a graph G and $k \in \mathbb{N}$

Question: Can we properly color the graph G using k colors

While there is a $\mathcal{O}^*(2^k)$ algorithm, we can see an interesting and simple $\mathcal{O}^*(3^k)$ algorithm.

Consider all the subset of vertices and store all the independent sets (that means can be 1 colored). Now consider subsets of vertices and store all the ones that can be partitioned into independent sets (which will take 3^n time as we are basically 3 partitioning the vertices into $S = X \sqcup Y$ and $V \setminus S$) giving us the 2-colorable subsets. Repeat with 3 coloring and so on. This will give an $\underbrace{\mathcal{O}^*(2^n) + \mathcal{O}^*(3^n) + \mathcal{O}^*(3^n) + ... + \mathcal{O}^*(3^n)}_{k \text{ terms}} = \mathcal{O}^*(3^n)$.

5. Kernalization

TODO. *Introduction with sorting and cycle detection idea*

Example : European Rail Network Problem

Given an input of $1,40,000$ trains, $25,000$ stations and $16,00,000$ single train stops (valid trains and station pairs).

Our question is if there is any subset T of trains, they wanted to find a smallest set S of stations such that each train T stops at atleast one of the stations in S.

This is roughly equivalent to the red-blue dominating set which is NP Hard. However, Karsten Werhe gave a set of reduction rules (similar to the cycle detection) that would split this into small cases which he could solve by hand.

TODO. *Section 4.6 of Fundamentals of Parametrized Complexity by Downey and Fellow*

This is a weird place as if we had a polynomial algorithm that could reduce the size of an NP problem by even 1 bit, we could just use the same algorithm to keep reducing it to 1 single bit (which would be the answer).

So what do we do?

TODO. *Parametrization of Kernel*

For vertex cover, we can use the following reduction rules:

- If there is a vertex v of degree zero in G, then delete v from G to get G'. Output: (G', k)
- If there is a vertex v if degree $\geq k+1$ in G, then delete v from G to get G'. Output: $(G', k-1)$

Let $\left(\hat{G}, \hat{k}\right)$ be an instnce to which none of the reduction rules applies. We then brute force from there.

We now need to prove:
1. Yes $\iff$ Yes
2. $|\hat{G}| + \hat{k} \leq f(k)$ for some computable function of k.

Proof.

TODO.

■

Definition: Kernalization

Let $L \subseteq \Sigma^* \times \mathbb{N}$ be a parametrized language. A **kernalization** algorithm (also called **reduction to problem kernel**) for L is an algorithm that given an instance (x, k) runs in time polynomial in $(|x| + k)$ and outputs an instance (x', k') such that
1. $|x'| + k' \leq g(k)$ for some computable g
2. $(x, k) \in L \iff (x', k') \in L$

(x', k') is called a kernel of L of size $g(k)$.

First Theorem of Parametrized Complexity 5.1. *A computable parametrized language L has an FPT algorithm if and only if it has a kernalization algorithm.*

Proof. Let there be a $O(f(x))$ (not necessarily polynomial) algorithm for the decision problem where x is the size of problem. ($\Longleftarrow$) From the definition of kernalization, we can find the kernel in $O(\text{poly}(|x|+k))$ time and then the solution in time bounded by $O(f(|x|+k)) \leq O(f(g(k)))$ which has no factor of n and hence, is an FPT.

($\Longrightarrow$) Suppose $\mathcal{A}$ solves instance (x,k) of L in time $O(f(k)\cdot|x|^c)$.

Algorithm 5.2. Run $\mathcal{A}$ on (x,k) for $|x|^{c+1}$ steps
- If $\mathcal{A}$ stops and returns an answer, return a trivial Yes or No instance
- Else: return (x,k)

It is clear that Yes $\iff$ Yes. What about the "$|x'|+k' \leq g(k)$ for some computable g"?

Well, as $\mathcal{A}$ is an FPT with runtime $O(f(x)\cdot|x|^c)$, the fact the algorithm has not terminated tells us:

$$\begin{aligned} |x|^{c+1} &= O(f(k)\ |x|^c) \\ \Rightarrow |x|^{c+1} &\leq c_2 f(k)\ |x|^c \\ \Rightarrow |x| &\leq c_2 f(k) \\ \Rightarrow |x| &= O(f(k)) \end{aligned}$$

Thus, $|x|+k = O(f(k)+k)$.

Thus, we have a kernel of size $O(f(k)+k)$. ∎

Notice, the FPT doesn't give a particularly useful kernalization. It is just a nice statement to say.

5.1. Planar Independent Set

Definition: Independent Set

Input: Given a graph G and $k \in \mathbb{N}$

Question: Does there exist $S \subseteq V$ such that $|S| \geq k$ such that $u,v \in S \iff uv \notin E$?

While in the classical setting, this and vertex cover are twins; The same doesn't track here. If we reduce to vertex cover, we have a $\mathcal{O}^*\left(2^{n-k}\right)$ algorithm which is not FPT.

Sadly, there is no FPT using the standard parametrization as the problem is $W[1]$ hard with respect to k.

However, if we restrict to the planar case, we can however have a kernel.

Definition: Planar Independent Set

Input: Given a planar graph G and $k \in \mathbb{N}$

Question: Does there exist $S \subseteq V$ such that $|S| \geq k$ such that $u,v \in S \iff uv \notin E$?

While this NP Hard (even on bounded degree), we now have a linear kernel (and hence, FPT) on the standard parametrization: if $|V| \geq 4k$ then by 4 color theorem, return **Yes**. Otherwise, return (G, k).

This is sometimes called a *cheat* kernel as we don't really find out much about the problem.

6. Feedback Vertex Set

Definition: Feedback Vertex Set

Input: Given an undirected graph G on n vertices, $k \in \mathbb{N}$.

Parameter: k

Question: Does G contain a set S of size $\leq k$ such that $G - S$ has no cycles.

We call $S \subseteq V(G)$ is a feedback vertex set of G if and only if $G - S$ is acyclic.

The idea is similar to vertex cover. We first delete all degree 0 and 1 vertices.

Now that we have a graph with degree atleast 2, there must either be a

TODO.

Theorem 6.1. *If* $n < (2\log_2 n)^k$ *then:*

$$(2\log_2 n)^k < (4k\log_2 n)^k$$

TODO. hain

Proof.

TODO. *hain*

■

7. Feedback Arc Set

Definition: Feedback Arc Set in Tournament

Input: Given a tournament T on n vertices, $k \in \mathbb{N}$

Parameter: k

Question: Does T contain an arc set F of size $\leq k$, such that $T - F$ has no directed cycle?

Such a F is called a feedback arc set of tournament T.

We could begin with getting rid of all the clear winners (out-degree 0) and clear losers (in-degree 0).

Now the tournament is guaranteed to have a cycle, we use the following claim:

Claim 7.1. An arc (and vertex) in a tournament is part of directed cycle if and only if it is part of a directed triangle.

Proof. FTSOC let the arc uv be part of a the shortest cycle c which is not a triangle.

TODO.

■

Similar to vertex cover, we would like to sort of have a rule to deal with arc's which are part of large number of triangle (k). But we can't delete it as otherwise the instance is no longer a tournament. So what do we do?

Definition

For a subset F of arcs of T, let $T \oplus F$ be the tournament obtained from T by reversing the arcs in F

Claim 7.2. If $T \oplus F$ is acyclic then F is a feedback set of T

Claim 7.3. If F is an inclusion minimal feedback arc set of T then $T \oplus F$ is acyclic.

TODO. *proof is hw*

Corollary 7.4. *If (T, k) is an Yes instance of FAST $\iff$ there is a set F of arcs of T, $|F| \leq k$, such that $T \oplus F$ is acyclic.*

These gives us clear reduction rules (which are very similar to vertex cover):

- **Reduction Rule 1**: If T has vertex v which is not part of any triangle in T, return $(T - v, k)$
- **Reduction Rule 2**: If T has an arc e that is part of $\geq k + 1$ triangles, then return $(T \oplus \{e\}, k - 1)$

TODO. *Prove Validity of rule 2!*

But why is this a valid kernel?

Proof. If both the reduction rules fail and we are in a non-trivial instance, notice that every arc is part of atmost k triangles and every vertex is part of a triangle.

Hence, each arc covers atmost k triangles. Thus, reversing an edge can 'destroy' atmost k triangles which cover $k + 2$ vertices.

Thus, if the instance has $> k(k + 1)$ vertices, we can return a No instance. ■

This gives us a $k^2 + 2k + k = O(k^2)$ kernel.

Idea

The usual idea in coming up with kernalization algorithms is to come up with reduction rules based on 'simple' cases.

The way we come up with these rules is to find useful structures we can discover in polynomial time.

Some common structures are:
- Something that is definitely part of the solution
- Something that is definitely not part of optimal solution
- Something which we can replace with a smaller/simpler thing

Note, this same idea gives us a $\mathcal{O}^*(3^k)$ FPT algorithm by branching on the triangles. $\mathcal{O}^*\left(2^{O(\sqrt{k}\log(k))}\right)$ is the best known bound for FAST by Alon, Lokshtanov and Saurabh in 2009.

For FVS, the easy bound is $\mathcal{O}^*(3^k)$. The iterated compression method gives $\mathcal{O}^*(2^k)$ and the current best is $\mathcal{O}^*(1.618^k)$ by Kumar[1] and Lokshtanov in 2016.

8. Feedback Vertex Set Revisited

Once we make the reductions and have a graph G left with degree atleast 3, consider that if X is a FVS and $G-X$ is a huge forest, then **every leaf in $G-X$ has to have a atleast 2 edges entering X.**

So we would like atleast some high degree vertices in the X. We could formalize this as:

Lemma 8.1. *Let $V(G) = \{v_1, v_2, ..., v_n\}$ in non-increasing order of degree.*

Let V_{3k} be the first $3k$ vertices in this order (with ties broken arbitrarily). Then, any FVS of G of size $\leq k$, must contain atleast one of the vertex of V_{3k}

Proof.

Claim 8.2. Let X be any FVS of G. Then,

$$\sum_{v \in X} (\deg(v) - 1) \geq m - n + 1$$

Proof. # edges in $G - X \leq n - |X| - 1$

other kind of edges $\leq \sum_{v \in X} \deg(v)$

Adding these, $m \leq \sum_{v \in X}(\deg(v) - 1) + n - 1 \Rightarrow \sum_{v \in X}(\deg(v) - 1) \geq m - n + 1$. ■

FTSOC, let $V_{3k} \subseteq V(G - X)$. Then,

[1] Another CMI person!

$$\sum_{v \in V_{3k}} (\deg(v) - 1)$$
$$\geq 3\left(\sum_{v \in X} (\deg(v) - 1)\right)$$
$$\geq 3(m - n + 1)$$

Consider

$$\sum_{v \notin V_{3k}} (\deg(v) - 1) \geq \sum_{v \in X} (\deg(v) - 1)$$
$$\geq m - n + 1$$

$$\sum_{v \in V(G)} (\deg(v) - 1) \geq 4m - 4n + 4$$
$$2m - n \geq 4m - 4n + 4$$

TODO. *Clarify?!*

Which is a contradiction. ■

This gives us a $\mathcal{O}^*\left((3k)^k\right)$ algorithm.

We can also have a randomized $\mathcal{O}^*\left(4^k\right)$ algorithm.

TODO. *Photo from Image*

9. Crown Decomposition

Notice, vertex cover was a high degree rule. It worked on high degree vertices. We might also note:

- Degree 0 rule $\rightsquigarrow$ min degree 1 graph: # vertices in the resulting kernel $\leq k + k^2$
- Degree 1 rule $\rightsquigarrow$ min degree 2 graph: # vertices in the resulting kernel $\leq k + \frac{k^2}{2}$
- Degree 2 rule $\rightsquigarrow$ min degree 3 graph: # vertices in the resulting kernel $\leq k + \frac{k^2}{3}$
- $\vdots$

Definition: Matching

A Matching M in Graph G is a set of edges in G of which no two share a common end-point.

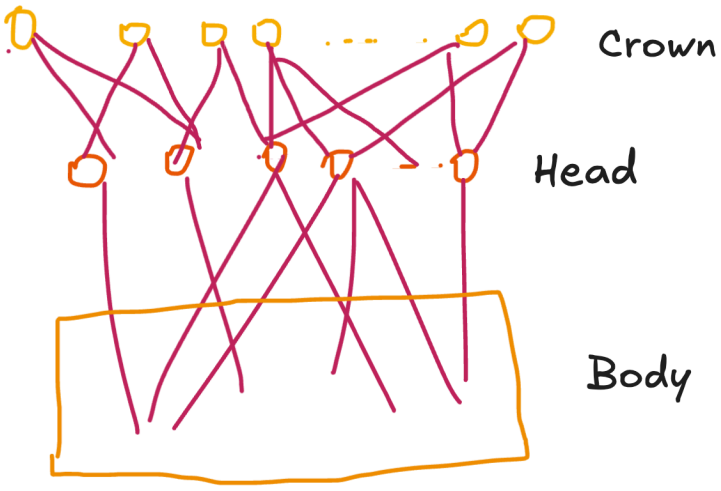
A vertex subset $X \subseteq V(G)$ is **saturated** by matching M is every vertex in X has an edge in M incident to it.

There is a (very, very non-trivial) polynomial time algorithm to find a maximal matching in a graph.

Definition: Crown Decomposition

A **Crown Decomposition** of a graph G is a of its vertex set $V(G)$ into three parts: $V(G) = C \uplus H \uplus B$ where:

1. The **crown** C is non-empty independent set in G
2. The set E' of edges between C and the **head** H contains a matching that **saturates** H.
3. There are no edges with one end-point in C nd the other in the **body** B.



Claim 9.1. For vertex cover, if $\langle C, H, B\rangle$ is a crown decomposition of a graph G then there is a smallest vertex cover of G that contains **all** of H and **none** of C.

Proof. As there is a matching, **not taking all** of H and taking **none** of C will leave some edge where both it's neighbors are not in the vertex cover.

If we don't take all of H and take **some** of C, we can have an equal or better instance by moving the 'guard' from a choosen C to the H it neighbors (can't happen that this doesn't occur as otherwise we have an edge that is not covered). ■

But to do any of that, we need to find the crown-decomposition which can't be polytime (unless $P = NP$). So what do we do?

Konnig's Theorem 9.2. *There is a smallest vertex cover of a bipartite graph B, is equal to the size of a maximum matching of B.*

There is a polynomial time algorithm that finds a largest matching, and a smallest vertex cover, on the input bipartite graph B

We now have an(other) algorithm for vertex cover.

A $\mathcal{O}^*(3k)$ kernel for Vertex Cover

1 Find maximal matching M of G.
2 ⌞ If $|M| > k$ then: return **No**
3 Let V_m be the set of all vertices involved in M. Then $I = (V(G) \setminus V_m)$ is an independent set.
4 ⌞ If $|I| \leq k$ then return (G, k)
5 So: $|I| > k$. Let B be the bipartite graph induced by $V_m \uplus I$. Find a largest matching $\widetilde{M}$ and a smallest vertex cover $\tilde{S}$ of B.
6 ⌞ If $|\widetilde{M}| > k$ then return **No**
7 Let C be the set of vertices in I that are **not** in $\tilde{S}$, H be the set of vertices of V_m that **are** in $\tilde{S}$ and let $B = V(G) \setminus (C \cup H)$ be the body.
8 ⌞ return $((G \setminus C \cup H), k - |H|)$

Our penultimate step indeed choose a crown decomposition as:
1. C is non-empty and independent (as subset of I)
2. H is saturated by a matching from C (otherwise, violates vertex cover property)
3. There are no edges with one end-point in C and the other in **body** B (otherwise, violates vertex cover or induced bipartite).

Remark

There is a linear programme to find a crown decomposition $\langle C, H, B\rangle$ such that $G \setminus C \cup H$ has no crown decomposition.

Furthermore, if $\langle C_1, H_1, B_1\rangle, \langle C_2, H_2, B_2\rangle, ..., \langle C_k, H_k, B_k\rangle$ are the crown decompositions of $G, G \setminus C_1 \cup H_1, ..., G \setminus (C_1 \cup C_2 \cup ... \cup C_{k-1}) \cup (B_1 \cup B_2 \cup ... \cup B_{k-1})$ respectively, then $\langle \bigcup C_i, \bigcup B_i, G \setminus (\bigcup C_i) \cup (\bigcup B_i)$ is a crown decomposition.

10. An $\mathcal{O}^*(2k)$ Kernel for Vertex Cover

This will be 'optimal' in some sense as Unique Game Conjecture would be violated by, $2 - \varepsilon$ approximation and thus, a kernel better than $(2 - \varepsilon)k$ is not possible.

We will proceed via ILP.

Definition: ILP of Vertex Cover

A variable x_v for each vertex $v \in V(G)$.

For each $uv \in E(G) : x_u + x_v \geq 1$.

For each $v \in V(G)$, $x_v \in \{0, 1\}$

Minimize $\sum_{v \in V(G)} x_v$ subject to above.

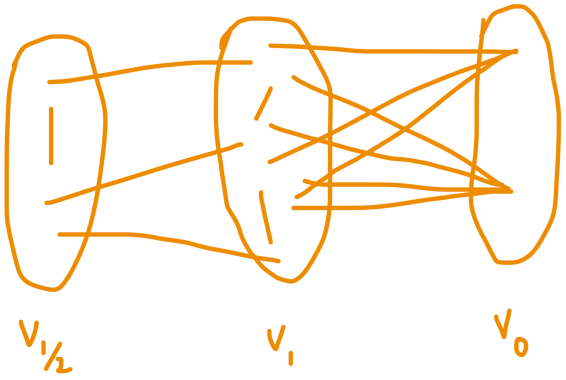
We can relax the integral constraint to get a linear programme

$$0 \leq x_v \leq 1 \text{ for all } v \in V(G)$$

As we shall see in sometime (and blackbox for now), vertex cover's LP has the half-integrability property.

Definition: Half-Integrability

If an ILP admits an optimal solution where every variable takes one of the values $\{0, \frac{1}{2}, 1\}$.



This sort of looks like a crown decomposition already...

Claim 10.1. There is a matching saturating V_1 from V_0

Proof. FTSOC, let there be no such matching. Then, by Hall's marriage lemma, there exists a $X \subseteq V_1$ such that $|N(X)| < |X|$ but then, reassign all variables in $N(X) \cup X$ as $\frac{1}{2}$ and that reduces the objective function.

This doesn't violate constraints as

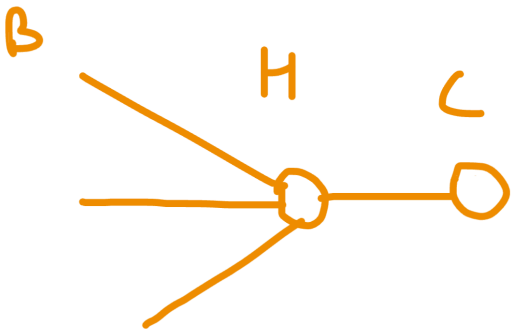
- $X \longleftrightarrow N(X)$ as $\frac{1}{2} + \frac{1}{2} = 1$
- $X \longleftrightarrow \frac{V_1}{2}$ as $\frac{1}{2} + \frac{1}{2} = 1$
- $N(X) \longleftrightarrow V_1 \setminus X$ as the vertex from $V_1 \setminus X$ was already 1 and $\frac{1}{2} + 1 > 0 + 1 \geq 1$.

Thus, we have a contradiction as the solution was optimal. ■

Notice, $V_{\frac{1}{2}}$ has less than $2k$ vertices as otherwise the relaxed instance has objective greater than the ILP which implies the ILP doesn't admit a solution. Thus, we already have a $\mathcal{O}^*(2k)$ kernel.

11. Sunflower Lemma

Notice, crown decomposition is sort of a generalization of the degree one rule.



Similarly, the sunflower lemma is a generalization of the large degree rule.

Definition: Sunflower

Let $\mathcal{U}$ be a finite universe and let $S_1, S_2, ..., S_t$ be subsets of $\mathcal{U}$ when:
1. There is a (possibly empty) set C such that $(S_1 \cap S_2) = C$ holds for all $1 \leq i \leq j \leq t$ and
2. $P_i = (S_i \cap C)$ is non-empty for each $1 \leq i \leq t$.

then $S_1, S_2, ..., S_t$ is. sunflower with core C and petals $P_1, P_2, ..., P_t$.

Erdos had asked Rado (who was a kid then, literally eating cereal): Fix positive integers k and d. Does just taking a large enough collection of distinct d-sized subsets of $\mathcal{U}$, guarantee a sunflower with k petals?

Rado answered shortly after leading to the **Sunflower Lemma**.

Theorem (Sunflower Lemma, Erdos & Rado 1960) 11.1. *Let $\mathcal{A}$ be a family of d-sized sets (without duplicates) over a finite universe $\mathcal{U}$. If $|A| > d!(k-1)^d$ then $\mathcal{A}$ contains a sunflower with k petals. Such a sunflower can be computed from $\mathcal{A}$ in time polynomial $|\mathcal{A}| + |\mathcal{U}| + k$.*

12. Iterated Compression

TODO. *Didn't go to class as sick and sleepy...*

A $\mathcal{O}^*(g(k))$ algorithm for the disjoint version $\Rightarrow$ an $\mathcal{O}^*\left(\sum_{i=0}^{k}\binom{k+i}{i}g(k-i)\right)$ algorithm for the original problem when $g(k) = \alpha^k$. This gives $\mathcal{O}^*\left((\alpha+1)^k\right)$.

13. Tournament Feedback Vertex Set

Definition: Tournament Vertex Set

Input: Given a tournament $\mathcal{T}$ on n vertices.

Parameter: k

Question: Is there a set $S \subseteq V(\mathcal{T}), |S| \leq k$, such that $T - S$ is acyclic?

TODO. *What?!*

14. Randomized Algorithms

A randomized algorithm can be thought of as a classical algorithm with access to a stream of random bits. If to solve a problem, the algorithm reads r random bits, then we measure it's success over the 2^r possible bit strings.

In the FPT world, most of the random algorithms we'll see will be **One Sided error Monte Carlo algorithm with false negatives**.

Definition: One Sided error Monte Carlo algorithm with false negatives

A random algorithm which always terminates in some bounded time and reports **NO** on all **NO** instances and **YES** on an **Yes** instance with probability $p \in [0, 1]$.

The reason we want one sided error is, say $p = \frac{1}{f(k)}$ for computable function f. Then if we repeat the algorithm $f(k)$ times.

Then the probability that the algorithm gives a wrong (**NO**) every time is $(1-p)^{\frac{1}{p}} \leq \frac{1}{e}$. Thus, we have a constant error odds at the end.

14.1. Color Coding

Definition: Hamiltonian Path

Input: Given a graph G on n vertices

Question: Does G have a (simple) path on n vertices

Definition: Simple Path

A (simple) path has no repeated vertices or edges.

This problem is known to be NP hard.

A weaker(?) version of this problem is

Definition: c-Path

Input: Given a graph G on n vertices and a constant c, independent of the graph.

Question: Does G have a path on c vertices?

This can clearly be solved in $\mathcal{O}(\binom{n}{c}c!) \approx \mathcal{O}(n^c c!)$ which is polytime. So at some point in between of constant and everything, we slip into NP.

Definition: k-Path

Input: Given a graph G on n vertices and a constant k, which could be dependent of the graph.

Question: Does G have a path on k vertices?

Monien in 1985 showed that k-path can be solved in $\mathcal{O}^*(k!)$ time and hence, $k = (\log(n))(\log(\log(n)))$ in P.

Papedmirou and Yennakekku in 1993 conjectured that k-path is in P for $k = o(\log(n))$. This was proven by Alon, Yuster and Zwick in 1994 using this new technique called **Color Coding**.

The problem with long path finding is that you sort of need to look at the path to find it. If I tell you that a graph has a small vertex cover, there is some structural property (sparseness) we have implied. Same with feedback vertex set. The issue is that I can take any random graph and affix a long path to it. This is what Alon, Yuster and Zwick had to deal with…

To simplify this problem:

Definition: Rooted k-Path

Input: Given a graph G on n vertices, a specified starting vertex s and a constant k, which could be dependent of the graph.

Parameter: k

Question: Does G have a path of length k that starts at s?

A naive algorithm could be to recurse on the neighbors of s as:

$$\text{SOL}(G, s, k) = \bigvee_{v \in N(s)} \text{SOL}(G - \{s\}, v, k-1)$$

We can get slightly better by DP. Consider a DP table D where D has one row for each vertex v in G and one column for each possible length $1 \leq i \leq k$.

$D[v, i]$ stores all paths of length i from s to v with s and v included.

TODO. *?*

This leads to a $\mathcal{O}\left(\binom{n}{k}nk\right)$ size table.

We make the following simplification: each vertex of G has one of the k colors $c_1, c_2, ..., c_k$.

Given this, look for a path that:

- starts at s
- has length k, and

- has all the k different colors

We can solve this using the above DP by storing the subset of colors that has already been seen.

This solves our problem in $\mathcal{O}\left(2^k nk\right) = \mathcal{O}^*\left(2^k\right)$ time.

But how do we do this coloring? Well, **randomly**!

Idea

If we randomly color the graph with the probability of vertex v being colored c_i being $\frac{1}{k}$ (uniformly).

Then, the odds of a k path having different colors is $\frac{k^k}{k!} \geq e^{-k}$ as $e^k = 1 + k + \frac{k^2}{2!} + ... + \frac{k^k}{k!} + ... \Rightarrow e^k \geq \frac{k^k}{k!}$.

Thus, if we repeat the above with a random coloring, we get a constant probability random algorithm with runtime $\mathcal{O}^*\left((2e)^k\right)$.

But these people didn't stop. We are using too much randomness as any subgraph of size k in graph can be detected in this manner. Why be this general?

Definition: Perfect Hash Family

A set $\mathcal{F}$ of functions from $\{1, 2, ..., n\}$ to $\{1, 2, ..., k\}$ is said to be a **(n, k) perfect hash family** if, for any subset $S \leq [n]$ of size k, there is a atleast one function $f \in F$ such that f is injective on S.

Theorem (Alan et al) 14.1. *For ay $n, k \geq 1$ one can construct an (n, k) perfect hash family of size $e^k k^{O(\log k)} \cdot \log(n)$ in time $e^k k^{\mathcal{O}(\log k)} \cdot n \log n$.*

We could just make the family and run the colorful path algorithm on the colorings induced by this family. This would still give an (un-randomized) algorithm in $\mathcal{O}^*\left((2e)^k\right)$.

In the 1993 paper by Papedmirou and Yennakekku, all open problems have been solved; other than one.

TODO. *Arvind and others have work on it to show if*

14.2. Random Separation

Cyan et al came up with the idea to perhaps use only 2 colors: one for the object we are trying to find and the other to separate it.

Idea

If we want to find an object X inside an input instance and we color the entire instance with 2 colors (say red and green) uniformly at random, then the probability that "our" object gets colored with green is $\frac{1}{2^{|X|}}$. This is "fpt" is $|X| \leq f(k)$

Definition: Subgraph Isomorphism

Input: Given graphs G, H where $|V(H)| = k, |V(G)| = n$

Parameter: k

Question: Is H a subgraph of G?

Definition: Clique

Input: Given a graph G and integer h where $|V(G)| = n$

Parameter: h

Question: Is there a h-clique as a subgraph of G?

Both of these are not expected to be in FPT as they are W[1] hard (Hardness of Clique $\Rightarrow$ Hardness of Subgraph Isomorphism)

However, if the degree of G is bounded, we can solve it using **random separation**!

Definition: Subgraph Isomorphism with degree bounded

Input: Given graphs G, H where $|V(H)| = k, |V(G)| = n$ and $\max_{v \in V(G)} \deg(v) = d$

Parameter: k, d

Question: Is H a subgraph of G?

1. Color every edge of G green or red, with probability $\frac{1}{2}$ each.
2. let $\chi : E(G) \to \{\text{red}, \text{green}\}$ be the resulting coloring
3. Let H' be an (unknown) copy of H inside G. Let Y be the set of edges that are: incident on a vertex of $V(H')$ and are not in $E(H')$.

Can Y have edges, both of whose end-points are in $V(H')$? Yes! These are the edges not in $V(H')$ as H is not necessarily a clique.

Say χ is a good if every edge of $E(H)$ is green and every edge in Y is red.

Claim 14.2. If H is connected, then H' will be equal to one of the connected components of the 'green' subgraph defined by a good coloring χ.

So we can delete the red edges and then look at the k sized connected components.

$$\mathbb{P}(\text{a random coloring is good}) = \frac{1}{2^{|E(H')|} \cdot 2^{|Y|}}$$

as we want all edges of H' to be green and all edges in Y to be red. As the degree is bounded and $E(H')$ and Y are incident on $V(H')$. Thus, $E(H') + |Y| = kd$. Thus,

$$\mathbb{P}(\text{a random coloring is good}) \geq \frac{1}{2^{kd}}$$

Thus, we can solve the problem (for connected H) by doing a uniform random coloring of edges to red or green, deleting red edges, checking connected components of size k for isomorphism and repeating if failed.

Isomorphism takes $O(k!k)$ time (brute force) or $O\left(k^{O(d \log d)}\right)$ as bounded degree (from Luki 1982).

We can extend this to a disconnected $H = H_1 \cup H_2 \cup ... \cup H_m$. After deleting the red edges, look for a component to match to (is isomorphic to) H_1, then H_2 and so on.

Theorem 14.3. *There are Monte Carlo algorithms with false negatives that solve bounded degree subgraph isomorphism in $\mathcal{O}^*\left(2^{dk}k!\right)$ and $\mathcal{O}^*\left(2^{dk}k^{\mathcal{O}(d \log d)}\right)$*

TODO. *thank god for OCR (from photo on phone)*

We can also use this on graphs without bounded degree.

Definition: Cutting of q Connect Vertices

Input: Given a graph G on n vertices, $q, k \in \mathbb{N}$

Parameter: $q + k$

Question: Is there a set $X \subseteq V(G)$ with $|X| \leq k$ such that $G - X$ has a connected component with exactly q vertices?

The problem was introduced by Danial Marx in 2006 and is known to be W[1]-hard for q or k alone. Marx also gave a $\mathcal{O}^*\left((q+k)^{\mathcal{O}(q)}\right)$ algorithm in 2006.

Let Y be a connected component of size q in $G - X$.

Color all edges and a good coloring is such that all edges of Y are green and all of X to Y are red (everything else is immaterial). This gives a $\frac{1}{q^2+kq}$ odds of good coloring and hence a $\mathcal{O}^*\left(k^2 + kq\right)$ algorithm.

But we can be faster. Consider coloring all the vertices. A good coloring has the Y vertices be green and X be red. Then, we can delete the red-green edges and check for a connected component.

This gives the odds of a good coloring to be $\frac{1}{2^{q+k}}$ and hence, an algorithm in $\mathcal{O}^*\left(2^{q+k}\right)$ time.

Remark

This would probably be publishable in 2006 or 2007 but Marx didn't know about random separation and now this is a classic exercise in the textbook (and has no other reference).

14.3. Divide and Color

This idea tries to combine divide and conquer with color coding to get fast randomized Algorithms. We will consider it on the k-path problem with parameter k (the length of path we are looking for).

Let P be on l-path in G, with end points u, v.

Partition $V(G)$ into two parts L, R uniformly at random.

We hope that the **first** $\lceil \frac{l}{2} \rceil$ vertices of P are in L and the rest in R. This has probability $\frac{1}{2^l}$.

Now we recursively look for path of length $\lceil \frac{l}{2} \rceil$ and $\lfloor \frac{l}{2} \rfloor$ in $G[L]$ and $G[R]$.

But how do we know they will patch together? By changing the question!

For $X \subseteq V(G), l \in \{1, 2, ..., k\}$ and $u, v \in X$, let

$$D_{X,l}[u, v] := \text{True} \iff \text{there is an } l - \text{vertex path from } u \text{ to } v \text{ in } G[X]$$

This means, once we have $D_{V(G),k}$ then we can just iterate over all the pairs of vertices and if any is true, we are done.

However, that would be very slow… so instead we compute an approximate version $\tilde{D}_{x,l}$ where

$$\tilde{D}_{x,l}[u, v] = \text{True} \Rightarrow D_{x,l}[u, v] = \text{True}$$

$$D_{x,l}[u, v] = \text{True} \Rightarrow \tilde{D}_{x,l}[u, v] = \text{True with high probability}$$

Let (L, R) be a partition of X. Let A be an $|L| \times |L|$ matrix, B be a $|R| \times |R|$ matrix. We define $A \bowtie B$ to be the $|X| \times |X|$ boolean matrix where $[u, v]$ entry is True if and only if $u \in L$, $v \in R$ and there is edge between $x, y \in E(G)$ with $x \in L, y \in R$, $A[u, x] = B[y, v] = \text{True}$.

This means $\left(D_{L,\lceil \frac{L}{2} \rceil} \bowtie D_{R,\lfloor \frac{L}{2} \rfloor}\right)[u, v] = \text{True}$ if and only if $G[X]$ has an l path from u to v where first $\lceil \frac{l}{2} \rceil$ vertices are in L and the rest are in R.

This implies (but not the other way) $D_{X,l}[u, v] = \text{True}$.

Simple Random Path

```
1 def SRP(X, l):
2     If l = 1: return D̃_{X,l}[v, v] = True for all v ∈ X and "False" otherwise.
3     Partition X into L, R uniformly at random
```

Simple Random Path

```
4   | $\tilde{D}_{L,\lceil \frac{L}{2} \rceil} = \text{SRP}\left(L, \lceil \frac{L}{2} \rceil\right)$
5   | $\tilde{D}_{R,\lfloor \frac{L}{2} \rfloor} = \text{SRP}\left(R, \lfloor \frac{L}{2} \rfloor\right)$
6   | return $\tilde{D}_{X,l} = \tilde{D}_{L,\lceil \frac{L}{2} \rceil} \bowtie \tilde{D}_{R,\lfloor \frac{L}{2} \rfloor}$
```

The recursion is $T(l) = 2T\left(\frac{l}{2}\right) + n^c \Rightarrow T(l) = l \cdot n^c = \mathcal{O}(n^{c+1})$.

Wait $P = \text{NP}$? Not really. Notice that our algorithm's success is not guaranteed. Looking at the tree, it is $\left(\frac{1}{2^l}\right)^{\log l} = \frac{1}{2^{l \log l}}$.

This gives an runtime of $\mathcal{O}^*\left(2^{l \log l}\right) = \mathcal{O}^*\left(2^{k \log k}\right) = \mathcal{O}^*\left(k^k\right)$ which is a lot, lot worse than our original $\mathcal{O}^*\left((2e)^k\right)$ color coding algorithm.

So what do we do? Our main issue is that the randomization keeps happening. Perhaps, let's front load the randomization.

Fast Random Path

```
1   def FRP(X, l):
2   |   If l = 1: return $\tilde{D}_{X,l}[v, v] =$ True for all $v \in X$ and "False" otherwise.
3   |   Set $\tilde{D}_{X,l}[u, v] =$ False for all $u, v \in X$
4   |   Repeat $f(l, k)$ times:
5   |   |   partition $X$ into $L, R$ uniformly at random
6   |   |   $\tilde{D}_{L,\lceil \frac{l}{2} \rceil} = \text{FRP}\left(L, \lceil \frac{l}{2} \rceil\right)$
7   |   |   $\tilde{D}_{R,\lfloor \frac{l}{2} \rfloor} = \text{FRP}\left(R, \lfloor \frac{l}{2} \rfloor\right)$
8   |   |   $\widetilde{D'}_{X,l} = \tilde{D}_{L,\lceil \frac{l}{2} \rceil} \bowtie \tilde{D}_{R,\lfloor \frac{l}{2} \rfloor}$
9   |   |   $\tilde{D}_{X,l}[u, v] = \tilde{D}_{X,l}[u, v] \vee \widetilde{D'}_{X,l}[u, v]$ for all $u, v \in X$
10  |   return $\tilde{D}_{X,l}$
```

The idea is that (based on $f(l, k)$) that we have a massive tree but we only need one subtree to succeed.

Let's say success for a node is just partitioning well, rest of the work is the job of it's children.

The probability of success of an arbitrary node in one try is $\frac{1}{2^l}$. Thus, the probability of failure in $f(l, k)$ tries is $\left(1 - \frac{1}{2^l}\right)^{f(l,k)}$.

Choose $f(l, k) = 2^l \log(4k)$.

This mke the probability of failure to be $\leq \frac{1}{4k}$.

Thus, probability that some node of the k-node skeleton fails is $\frac{1}{4}$ by the union bound.

This, probability that none of the skeleton fails, that is our algorithm finds the path is $\frac{3}{4}$.

Let's now look at the running time.

$$\begin{aligned} T(l,k) &\leq 2^l \log(4k) 2T\left(\frac{l}{2}, k\right) + n^c \\ \Rightarrow T(l,k) &= 4^{l+o(l+k)} n^d \\ &< 4^{k+o(k)} n^d \\ &< 4^{k+o(k)} n^d \end{aligned}$$

This gives an $\mathcal{O}^*\left(4^k\right)$ algorithm.

The state of the art is $\mathcal{O}^*\left(1.618^k\right)$ where 1.618 is the golden ratio.

14.4. Chromatic Coding

We will be looking at a class of problems that is characterized as:

Definition 14.4. Let $\mathcal{F}$ be some family of graph (for example, say chordal, d-cluster, interval etc)

Input: Given a graph G on n vertices and $k \in \mathbb{N}$

Parameter: k

Question: Is there a set $A \subseteq \binom{V(G)}{2}$ of atmost k adjacencies of G such that $G \oplus A \in \mathcal{F}$?

where $G \oplus A = G'$ where if $e \in A$ and $e \in E(G)$ then $e \notin E(G')$ and if $e \in A$ and $e \notin E(G)$ then $e \in E(G')$ and the other edges remain as is.

The general algorithm involves: coloring $V(G)$ with q colors at random, such that (with good probability) for every $uv \in A$, $\chi(u) \neq \chi(v)$ for the coloring we get.

This partitions $V(G) = V_1 \cup V_2 \cup ... \cup V_q$ where $v \in V_i \iff \chi(v) = i$.

If $\mathcal{F}$ is closed under taking induced subgraphs, then $G[V_i]$ is also in $\mathcal{F}$. Note, this is true for our examples. We exploit this property to create our algorithm.

First we ask: how many colors do we need, to color A 'properly'?

1. An arbitrary graph with n vertices? n because of K_n
2. A planar graph on n vertices? 4, by 4-color theorem
3. A graph on n vertices and max degree Δ? $\Delta + 1$ by simple greedy coloring.
4. A graph on n vertices with min degree d? n by simply a clique with one out branch (leading to $d = 1$)

Definition: Degeneracy

Given a graph G, we say it has degeneracy δ if there is a v vertex with degree atmost δ and $G - v$ also has degeneracy δ.

For example: A tree has $\delta = 1$ as we can delete a leaf and still be left with a lead.

5. A graph on n vertices with degeneracy δ?

Solution

We form a chain $v_1, v_2, ..., v_n$ where v_i has degree $\leq \delta$ in $G - \{v_1, v_2, ..., v_{i-1}\}$.

Color greedily in reverse order: $v_n, v_{n-1}, ..., v_1$.

This won't lead to a conflict as v_k has atmost δ neighbors in $\{v_{k+1}, v_{k+2}, ..., v_n\}$.

This motivates:

Lemma 14.5. *Any graph G on n vertices with k edges has degeneracy $\leq \lceil \sqrt{2k} \rceil$.*

Proof. FTSOC, let the above not be true. It is clear that:

$$\text{Sum of degrees} \leq 2k > \delta n$$
$$\Rightarrow n < \frac{2k}{\delta}$$
$$\Rightarrow k \leq \binom{n}{2} < \binom{\frac{2k}{\delta}}{2}$$

As $\delta \geq \sqrt{2k}$,

$$k < \binom{\sqrt{2k}}{2} < k$$

which is absurd!

Thus, our initial assumption must be false. Thus, any graph G on n vertices with k edges has degeneracy $\leq \lceil \sqrt{2k} \rceil$. ■

A lemma we'll prove a bit later is

Lemma 14.6. *If the vertices of a graph H on $\leq k$ edges are colored uniformly at random with $q = \lceil \sqrt{8k} \rceil$, then the probability that H is properly colored is $\geq 2^{-\sqrt{\frac{k}{2}}}$.*

Definition: ℓ-cluster graph

A graph with $\leq \ell$ components with each being a clique is called a ℓ-cluster graph.

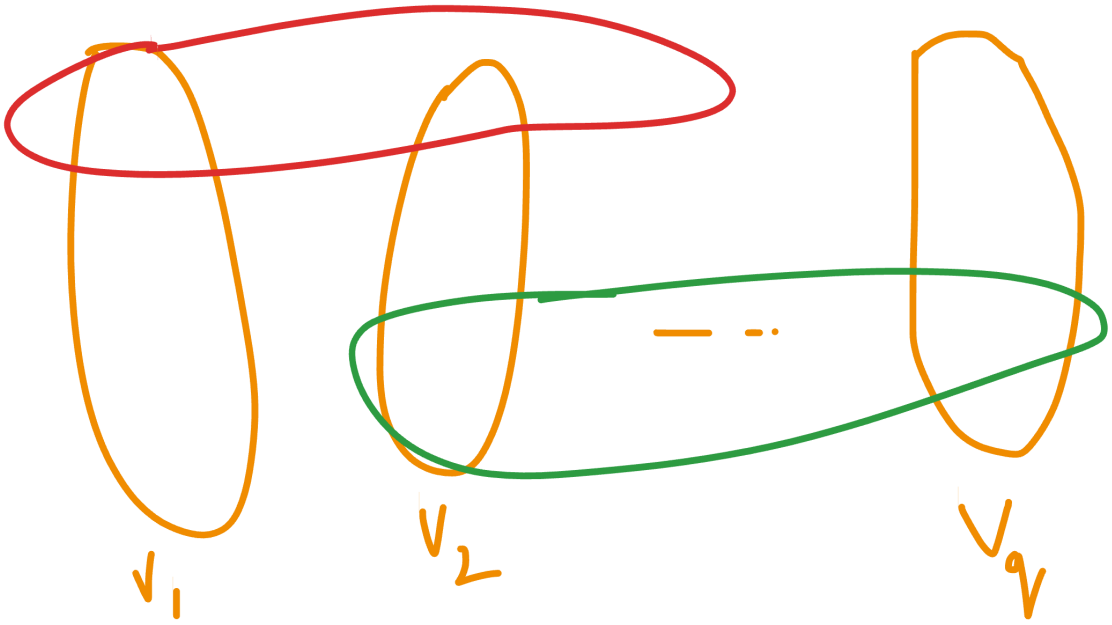
Definition: d-cluster

Input: Given graph G and $k \in \mathbb{N}$

Parameter: k

Question: Does there exist $A \subseteq \binom{V(G)}{2}$ and $|A| \leq k$ abd $G \oplus$ is a d-cluster?

After our coloring,



For say 2-clustering, the two clusters are divided among our vertexes.

We have $\leq 2q = \lceil 2\sqrt{8k} \rceil$ cliques across the color sets.

We can make a guess for the clusters position.

There are $2^{\mathcal{O}(\sqrt{k})}$ possible guesses and only 1 is correct.

This gives us a $2^{\mathcal{O}(\sqrt{k})}$ aka sub-exponential algorithm.

We can extend this for d-clustering.

This was solved by Alon, Lokshtanov and Saurabh where Lokshtanov and Saurabh found the randomized algorithm while Alon de-randomized it (as they wrote to him).

15. Graph Minors

Definition: Contraction of an edge

To contract an edge uv:
- Delete u and v
- Add a new vertex w adjacent to $(N(u) \cup N(v)) \setminus \{u, v\}$

We denote this as $G \setminus uv$

Definition: Minor

A graph H is a minor of graph G if $G \rightsquigarrow H$
- Vertex deletions
- Edge deletions

- Edge contractions

in any order. We denote this by $H \underset{m}{\succeq} G$.

Equivalently, H is a contraction of a subgraph of G.

Notice, we can characterize forests as graphs that don't have a triangle as a minor (as we can't introduce cycles). Sort of, some properties are preserved under taking a minor.

Exercise

Let $H \underset{m}{\succeq} G$. If G has the property ____ then H as has it:
1. No Edges - TRUE
2. Forest - TRUE
3. Tree - FALSE (delete vertex to make forest)
4. Connected - FALSE (see above)
5. $\Delta(G) \leq 3$ - FALSE
6. Bipartite - FALSE (take an even cycle, now contract an edge)
7. Planar - TRUE
8. $|\text{dominating}(G)| \leq 1$ - FALSE (delete the midpoint of the star)

Definition: Minor Closed

A family $\mathcal{F}$ of graphs is **minor closed** if $G \in \mathcal{F}$ and $H \underset{m}{\succeq} G \Rightarrow H \in \mathcal{F}$.

Definition: Obstruction Set

The obstruction set (also called "forbidden set") of a family $\mathcal{F}$ of graphs is the set of all graphs H such that
1. $H\neg \in \mathcal{F}$
2. every proper minor of H is in $\mathcal{F}$

This is denoted $\text{Obstr}(\mathcal{F})$

Lemma 15.1. *If $\mathcal{F}$ is minor-closed, then $G \in F$ if and only if no graph in $\text{Obstr}(\mathcal{F})$ is a minor of G.*

Proof.

TODO.

■

This implies that minor-closed families are characterized by the obstruction set.

Definition: Quasi Ordering

A binary relation which is reflexive and transitive is called a Quasi[2] ordering.

Ex: A is a subgraph of B or A is a minor of B

Definition: Well Quasi Ordering

A Quasi ordering ofer a set $\mathcal{U}$ is a well-quasi orderng (WQO) if **every infinite** sequence $\langle x_1, x_2, ... \rangle$ of elements of $\mathcal{U}$ contains atleast one pair x_i and x_j with:

$$i < \wedge$$

x_i succ.eq x_j

Notice, "is a subgraph of" is not a well-quasi ordering as in the sequence of cycles $C_1, C_2, ...$, no earlier graph is a subgraph of a latter graph.[3]

Robertson and Seymour proved, what was previously called Wagner's conjucture:

Theorem 15.2. *Finite Graphs are well-quasi ordered by the graph minor relation.*

This proofs were started in 1983 with the paper "Graph Minor I" and ended in 2004 with the paper "Graph Minor XX" (and expanded till "Graph Minor XXIII").

These papers used almost half the words of the English dictionary as terms and are a gold mine of ideas. It is belived that this was delayed due to the only qualified reviewer dying due to old age.

Corollary 15.3. *Every minor closed family of graphs has a finite obstruction set.*

Note, given G and H, checking if H minor of G is NP-Hard as we can check for C_n in a n-vertex graph which is NP-Hard as it solves Hamiltonian Path.

But if we get to fix H, then Robertson and Seymour showed:

Theorem 15.4. *There is a computable function f and an algorithm that, given graph H and G, check's if $H \underset{m}{\succeq} G$ in time $f(|H|) \cdot |V(G)|^3$*

Sadly, this is not constructive. Also, the f is a tower of power of 2's where the height is also a tower of power of 2's whose height depends on $|H|$. This makes their algorithm 'galactic' as the constant is massive even for small H.

Notice, as planar graphs are characterized by linkless embedding (some old paper), we can use the corollary to find the finite obstruction sets. We can then use Robertson and

[2]Can't call it Qusai ordering sadly.

[3]How did none of us figure this out till Prof. Philip pointed it out?!

Seymour's algorithm. We didn't even know the obstruction sets (which we now know to be 7).

This sort of gave us the proof of an algorithm existing without explictly finding it. But it does tell us that the problem is in Polytime.

16. I was home for like 2-3 classes

TODO.

17. DP over Subsets

TODO.

17.1. Set Cover

Definition: Set Cover

Given: A finite universe $\mathcal{U}$, a family $\mathcal{F}$ of subsets of $\mathcal{U}$ and $k \in \mathbb{N}$.

Question: Does there exist a subfamilly $\mathcal{F}' \subseteq \mathcal{F}$ with $|\mathcal{F}'| \leq k$ and $\bigcup \mathcal{F}' = \mathcal{U}$?

This probalem is unlikely to be FPT with k as it reduces to **Dominating Set** which is W[2]-hard (for parameter k).

Definition: Dominating Set

Given: Undirected graph G on n vertices and $k \in \mathbb{N}$

Parameter: k

Question: Is there a subset $S \subseteq V(G)$ of vertices, $|S| \leq k$ such that $N[S] = V(G)$?

We can reduce to set cover by $\mathcal{U} = V(G)$ and $\mathcal{F} = \{N[v] \mid v \leftarrow V(G)\}$.

But dominating set has an obvious FPT algorithm on n (just check all subsets!).

Does set cover have someting on $|\mathcal{U}| = n$? Yes. We can check all subsets of $\mathcal{F}$ in $2^{|\mathcal{F}|}$ time. As $|\mathcal{F}|$ is bounded by 2^n, we have a $O^*(2^{2^n})$ algorithm.

Can we do better?

Consider that the smallest set cover has $\leq n$ subsets from $\mathcal{F}$ (as each element is in atleast one subset). This means we only need to look at $\leq n$ sized sub-families. That gives an $O^*\left(\binom{2^n}{n}\right) = O^*\left(2^{n^2}\right)$ algorithm.

But can we do better?

Let $\mathcal{F} = \{F_1, F_2, ..., F_m\}$ for $X \subseteq \mathcal{U}$ and $0 \leq j \leq m$, let $\mathrm{dp}[X, j]$ denote the smallest size of a sub-family of $\{F_1, ..., F_j\}$ that covers X and $\mathrm{dp}[X, j] = \infty$ if there is no such family.

We have the base case $\mathrm{dp}[\emptyset, 0] = 0$ or $\mathrm{dp}[X, 0] = \infty$ if $X \neq \emptyset$.

For $j \geq 1$:

$$\mathrm{dp}[X, j] = \min\big(\mathrm{dp}[X, j-1], \mathrm{dp}\big[X \setminus F_j, j-1\big] + 1\big)$$

We can output $T[\mathcal{U}, m]$.

This has space $O^*(2^n \cdot m)$ and time $O^*(2^n \cdot m)$ which are both bounded by $O^*(4^n)$.

17.2. Steiner Tree

Definition: Steiner Tree

A Steiner Tree for $K \subseteq V(G)$ a connected subgraph H of G with $K \subseteq V(H)$.

Definition: Steiner Tre Problem

Given: Connected graph G with edge weights $w : E(G) \to \mathbb{R}_{\geq 0}$ and a set $K \subseteq V(G)$ of terminals.

Question: Find a Steiner Tree of K with minimum total weight.

For $|K| = 2$, this is just the shortest path. For $|K| = |V(G)|$, this is just the minimum spanning tree.

But for intermiddiate K, it is NP Hard. It is known to be NP-Hard for $|K| \geq 3$ for even unweighted graphs!

Why is this named Steiner Tree and not graph? As if it had cycles, we could delete edges to reduce weight.

Also, the leaves have to be terminal (but not vice versa) as otherwise, we could lose the non terminals leaf and have a Steiner Tree with lower weight.

Can we now see a $O^*\big(n^{f(k)}\big)$ algorithm?

Let H be a Steiner Tree on k. Guess $X \subseteq K$ that are leaves in H.

Lemma 17.1. *If a tree has k leaves, then it has atmost $k-2$ vertices of degree ≥ 3.*

Proof. By Handshaking Lemma and let vertices of number of degree $1, 2, \geq 3$ being $k_1 = k, k_2, k_3$:

$$\begin{aligned} k_1 + 2k_2 + 3k_3 &\leq 2(k_1 + k_2 + k_3 - 1) \\ \Rightarrow k_1 + 2k_2 + 3k_3 &\leq 2k_1 + 2k_2 + 2k_3 - 2 \\ \Rightarrow k_3 &\leq k_1 - 2 = k - 2 \end{aligned}$$

Thus, if a tree has k leaves, then it has atmost $k-2$ vertices of degree ≥ 3. ∎

Thus, H has $\leq |X| - 2$ vertices of degree ≥ 3.

We can hence, guess the large degree vertices in n^k time as well.

Theorem 17.2 (Cayley's Theorem)**.** *There are $\leq t^t$ trees on t vertices.*

Using above, we can hence solve our problem in $O^*\left(n^{f(k)}\right)$ time. Hence, problem in in XP.

Dreyphus-Wagner in 1971 gave an algorithm to solve this in $O^*\left(3^{|K|}\right)$ time.

Neederlof showed that $O^*\left(2^{|K|}\right)$ for unweighted graph via inclusion-exclusion.

17.2.1. An *Avent la lettre* FPT Algorithms

We may assume

1. $|K| \geq 2$ as otherwise, there is not much to do.
2. Every terminal has degree $= 1$ and it's unique neighbor is not a terminal. (just add a new leaf vertex attached to the terminal and make it the terminal!)

This implies we have been given a set of things that have to be leaves and want to find a minimum weight spanning tree.

Let H be an (unknown) optimal Steiner tree for k. Pick an arbitrary edge xy of H and delete it. We get two trees H_x and H_y .

Note, both these must have non-zero terminal nodes. Let them be D and $K \setminus D$ respectively.

Notice, H_x is an optimal Steiner tree for $D \cup \{x\}$ and H_y is an optimal Steiner Tree for for $(K \setminus D) \cup \{y\}$.

Thus, to find H, it is enough to find, for each $D \subseteq K$ and each $v \in V(G) \setminus K$ the optimal Steiner tree fr $D \cup \{v\}$. Save this as $T(D, v)$. We can now report:

$$\min_{\substack{xy \in G, \\ x,y \notin K \\ D \subseteq K}} T(D, x) + T(D \setminus K, y) + w(x, y)$$

Problem is that some v could have degree one in optimal steiner tree for $D \cup \{v\}$ for some D. This can't be computed by the DP table as we refer to different cardinality subsets in the recursion and can't use the same…

So we guess the first vertex u in the optimum Steiner Tree for $D \cup \{v\}$ where the tree branches. Do the 'tree splitting' at this vertex u and not at v (possibly $u = v$).

For each non-empty $D \subseteq K$ and each $v \in V(G) \setminus K$, let

$$T[D, v] = \text{the minimum weight Steiner Tree for } D \cup \{v\}$$

Base case:

$$T[\{t\}, v] = \text{dist}(t, v)$$

Reccurence:

$$T[D, v] = \min_{\substack{u \in V(G) \setminus K, \\ \emptyset \neq D' \subset D}} T[D', u] + T[D \setminus D', u] + \text{dist}(v, u)$$

Notice, we use $O^*\left(2^k n\right) = O^*\left(2^k\right)$ space.

To compute for $|D| = i$, we take atmost $n \cdot 2^i$ time. Thus, our time complexity is

$$\begin{aligned}
&\sum_{i=0}^{k} \left(n \cdot \binom{k}{i} 2^i \right) \\
&= n \sum_{i=0}^{k} \binom{k}{i} 2^i \\
&= n(2+1)^k \qquad \text{by Binomial Theorem} \\
&= n \cdot 3^k
\end{aligned}$$

Thus, we need $O^*\left(3^k\right)$ time.

18. Fast Subset Convalution

Definition: Subset Convalution

Given $N := \{1, 2, ..., n\}$ and $f, g : 2^N \to \mathbb{R}$; we define $f * g$ as:

$$(f * g)(S) := \sum_{T \subseteq S} f(T) g(S \setminus T)$$

Doing this naively would take $O^*(3^n)$ time.

Direct FFT gives $\sum_{T \subseteq N} f(T) g(S \setminus T)$ in $O^*(2^n)$ time.

Definition: Mobius Transform

Given $f : 2^N \to R$, the forward mobius transform (MT):

$$(f : S \mapsto f(s)) \underset{\text{MT}}{\to} \left(F : S \mapsto \sum_{T \subseteq S} f(T) \right)$$

and the inverset mobius transform is (IMT):

$$(f : S \mapsto f(s)) \underset{\text{IMT}}{\to} \left(F : S \mapsto \sum_{T \subseteq S} (-1)^{|S \setminus T|} f(T) \right)$$

Theorem 18.1. $f \underset{\text{MT}}{\to} F \underset{\text{IMT}}{\to} f$

The proof is in the ~~pudding~~ computation.

Directly computing the mobius transform would take $O^*(3^n)$ time.

But as it turns out, we can do it in $O(2^n \cdot n)$ time.

Consider $f = F_0, F_1, ..., F_n$ and

$$F_0(S) = f(S)$$
$$F_k(S) = \begin{cases} F_{k-1}(S) & \text{if } k \in S \\ F_{k-1}(S) + F_{k-1}(S - \{k\}) & \text{if } k \in S \end{cases}$$

We can show by induction that

$$F_k(S) = \sum_{T \subseteq S \cap \{1,2,...,k\}} f(T \cup (S \cap \{k+1, ..., n\}))$$

This can be computed in $O(2^n \cdot n)$ time. A similar reccurence gives $O^*(2^n \cdot n)$.

Definition: Ranked Mobius Transform

We define Ranked Mobius Transform (RMT) as:

$$(f : S \mapsto f(S)) \underset{\text{RMT}}{\longrightarrow} \left(F : (S, k) \mapsto \sum_{\substack{T \subseteq S \\ |T| = k}} f(T) \right)$$

and the Inverse Ranked Mobius Transform which goes from $(2^N \times N \to R) \to (2^N \to R)$ as:

$$(F : (S, k) \mapsto F(s, r)) \underset{\text{IRMT}}{\longrightarrow} \left(\sum_{T \subseteq S} (-1)^{S \setminus T} F(T, |S|) \right)$$

Note, RMT and IRMT are inverse for obvious reasons as IRMT is 0 only when $T = S$.

Definition: Adithya's Operator

We define $F \circledast G$ on ranked functions as:

$$(F \circledast G)(S, k) = \sum_{j=0}^{k} F(S, j) F(S, k - j)$$

We now are ready to compute $f * g$.

We first compute $f, g \underset{\text{RMT}}{\to} F, G$, then compute $F \circledast G$ and finally compute $(f \star g)(S) = \text{IRMT}(F \circledast G)(S)$.

Theorem 18.2. $(f * g)(S) = \text{IRMT}(F \circledast G)(S)$

Proof.

$$
\begin{aligned}
\text{RHS} &= \text{IRMT}(F \circledast G)(S) \\
&= \sum_{T \subseteq S} (-1)^{|S \setminus T|} (F \circledast G)(T, |S|) \\
&= \sum_{T \subseteq S} (-1)^{|S \setminus T|} \sum_{j=0}^{|S|} F(T, j) G(T, |S| - j) \\
&= \sum_{T \subseteq S} (-1)^{|S \setminus T|} \sum_{j=0}^{|S|} \left(\sum_{\substack{U \subseteq T, \\ |U| = j}} f(U) \right) \left(\sum_{\substack{V \subseteq T, \\ |V| = |S| - j}} f(V) \right) \\
&= \sum_{T \subseteq S} (-1)^{|S \setminus T|} \sum_{\substack{U, V \subseteq T, \\ |U| + |V| = |S|}} f(U) g(V) \\
&= \sum_{T \subseteq S} (-1)^{|S \setminus T|} \sum_{\substack{U, V \subseteq S, \\ |U| + |V| = |S|}} f(U) g(V) \delta_{U, V \subseteq T} \\
&= \sum_{\substack{U, V \subseteq S, \\ |U| + |V| = |S|}} f(U) g(V) \sum_{T \subseteq S} (-1)^{|S \setminus T|} \delta_{U \cup V \subseteq T} \\
&= \sum_{\substack{U, V \subseteq S, \\ |U| + |V| = S}} \sum_{U \cup V \subseteq T \subseteq S} (-1)^{|T|}
\end{aligned}
$$

which by inclusion-exclusion is $f * g$. ■

We can use this now to solve some problems.

18.1. Chormatic Number

Given $G = (V, E)$, we define $S \subseteq V$ as

$$
f(S) = \begin{cases} 1 & \text{if } S \text{ is independent} \\ 0 & \text{otherwise} \end{cases}
$$

Notice, $f * f$ being non-zero indicates 2-colorablity. Similarly, $f * f * f$ being non-zero indicates 3-colorability and so on.

Thus, we want to check $f^{*k}(V)$ is non-zero which takes $O^*(2^n)$ time.

We can also use this to find the chromatic number in $O^*(2^n)$ time (which is something we didn't know about till 2010).

18.2. Dreyphus-Wagner

Instead of doing convalution on the ring $(\cdot, +)$, we want to do it over the semi-ring $(+, \max \text{or} \min)$ that is:

$$
(f * g)(S) = \max_{T \subseteq S} (f(T) + g(S \setminus T))
$$

Let $\beta = 2^n + 1$

Consider $f'(S) = \beta^{f(S)}$ and $g'(S) = \beta^{g(S)}$.

Using Fast Subset Convalution, on f' and g', we have:

$$(f' * g')(S) = \sum_{T \subseteq S} \beta^{f(T)+g(S)}$$

This allows us to compute the max as in base β, there is no carryovers. So we can find the max position that is non-zero and we are done.

And thus, we can find the convalution on $(+, \max)$ in $O^*(2^n)$ time.

We will now use this to find the minimum steiner tree.

Let $G = (V, E)$ and $k \subseteq V$ is our target set

TODO.

19. Back to Steiner Tree!

We will now look at an algorithm that gives us steiner tree in $O^*(2^n)$ time.

This was found by Lindler and uses PIE to design an algorithm.

Idea

Given a target object $\mathcal{T}$ which is "hard" to locate.

Define a set $\mathcal{U} \geq \mathcal{T}$ of 'relaxed' objects that are easier to count.

Define "requirements" $A_1, A_2, ..., A_n \subseteq \mathcal{U}$ that satisfies $T \neq \emptyset \iff \cap A_i \neq \emptyset$.

Use $|\cap_{i\in[n]} A_i| = \sum_{x\in[n]} (-1)^{|X|} \, |\cap_{i\in X} (\mathcal{U} \setminus A_i)$.

19.1. Unweighted Steiner Tree

Definition: Unweighted Steiner Tree

Input: Connected graph G, $K \subseteq V(G)$ (terminals) with $|K| = k \geq 2$, integer l.

Given: $k = |K|$

Question: Is there a subtree H of G with $K \subseteq V(H)$ and $|E(H)| \leq l$?

The motivation is the way we relaxed the notion of a path to that of a walk, maybe a relaxation could help here.

Definition: Graph Homomorphism

A homomorphism from graph H to a graph G is any function $h : V(H) \to V(G)$ such that $xy \in E(H) \Rightarrow h(x)h(y) \in E(G)$.

Notice, a walk with l edges in a G is the image of homomorphism from a path with l edges into G.

Notice, homomorphism is preserving a local property in case of path and hence, it's easy to count!

$$\begin{aligned}&\text{number of walks in G that start at a and have l edges}\\ =&\sum_{b\in N(a)}\text{number of walks in G that start at b and have (l-1) edges}\end{aligned}$$

Linlorf obtained a similar formula for trees (although a bit more involved).

Definition: Ordered Trees

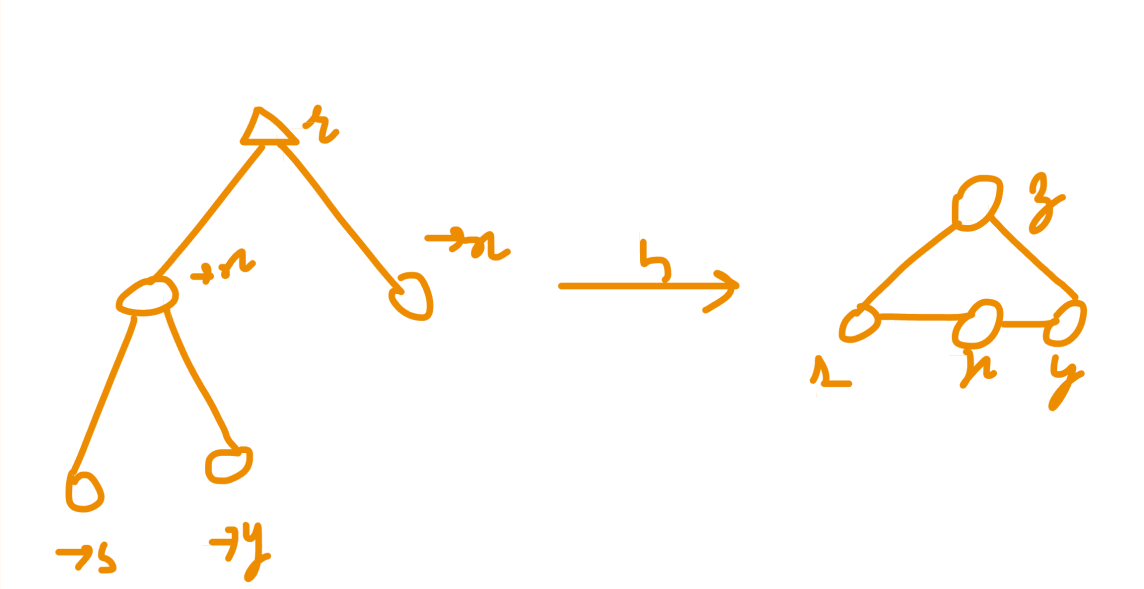
An ordered tree is a rooted tree in which the children of each node are ordered (first child, second child, so on ...)

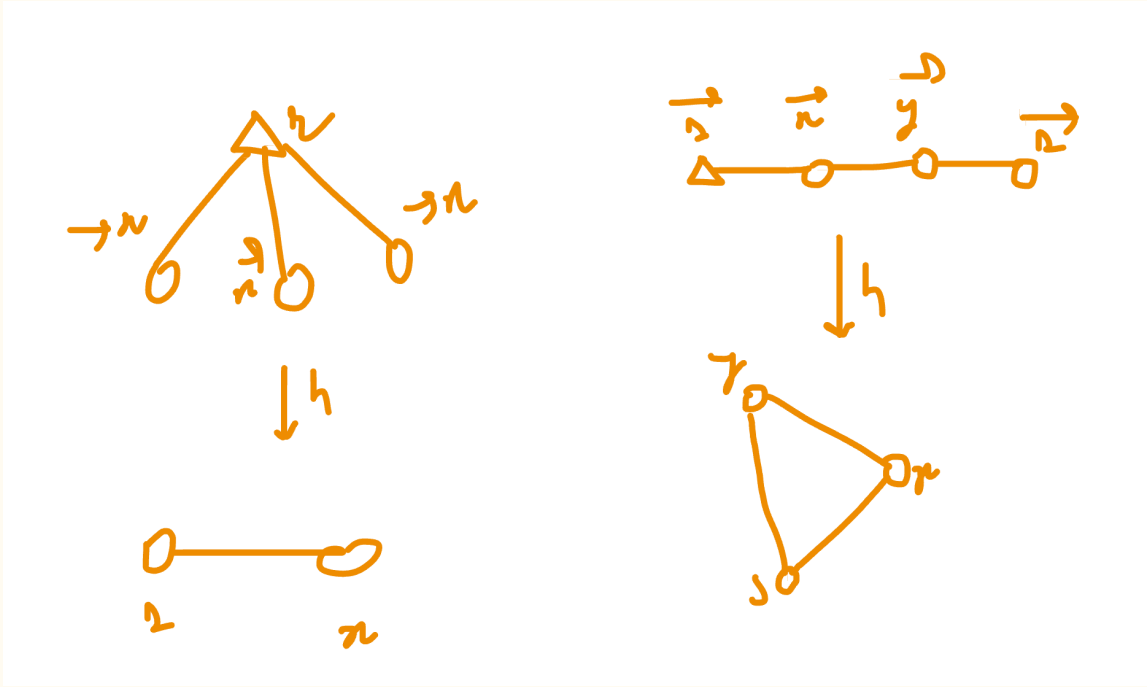
Definition: Branching Walk

A branchig walk in a graph G is a pair $B = (H, h)$ where H is an ordered rooted tree and $h : V(H) \to V(G)$ is a homomorphism

- B is a from vertex $s \in V(G)$ if $h(\text{root of H}) = s$
- The length of $B = |E(H)|$
- $V(B) = h(V(H))$

Example





Notice, the image is connected!

Claim 19.1. Let $k \in K$. G has H with $K \subseteq V(H)$ and $E(H) \leq l \iff G$ has a branching walk B from s with $K \subseteq V(B)$ and length of B is exactly l.

Proof. ($\Longrightarrow$) Identity map!

($\Longleftarrow$) Just look at the image of the mapping and find the spanning tree. This exists as the image is connected and thes spanning tree has $\leq l$ edges. ■

And now, we kill using PIE!

Fix a terminal $s \in K$. Let $k' = K \setminus \{s\}$.

Relexed objects $\mathcal{U}$: All branching walks of length l from s.

Requirements: For each $v \in K', A_v = \{B \in \mathcal{U}, v \in V(B)\}$.

We need to check: is $\cap_{v \in K'} A_v \neq \emptyset$

$$|\cap_{v \in K'} A_v| = \sum_{X \subseteq K'} (-1)^{|X|} \, |\{B \in \mathcal{U}; V(B) \cap X = \emptyset\}|$$

Fix $X \subseteq K'$. Let $G' = G - X$.

For $a \in V(G')$ and $0 \leq j \leq l$:

$$c_j(a) := \text{the number of branching walks of length j from a in G'}$$

We now use the fact we had numbered the children!

$$c_0(a) = 1$$
$$j \geq 1 : c_j(0) = \sum_{t \in N_{G'}(a)} \sum_{j_1 + j_2 = j-1} c_{j_1}(a) \times c_{j_2}(t)$$

Putting this together, we have steiner tree in $O^*(2^n)$ time and polynomial space.

20. Tree Width

This was one of the notions introduced in the 25 papers proving graph minor relation is well-quasi ordered. Although, similar ideas were already present in graph theory. It is roughly 'how close a graph is to a tree'.

We are interested in it as a lot of NP Hard problems are FPT on graphs with bounded tree-width. For example: (weighted) vertex cover is solvable in $O^*\left(2^{\text{tw}(G)}\right)$ and $\text{tw}(G) \leq \text{VC}(G) + 1$ and is in practice a lot smaller.

Hans Badleender used to publish papers with like 20-30 problems solved with tree-width in one paper. One could use 'tree-width' as his middle name.

At some point Bruno Courcelle got pissed off and decided to shut down this pipeline by proving

Theorem 20.1. *Every graph problem that can be expressed in* MSO_2 *as a formula* φ *can be solvd in time* $f(\text{tw}(G), |\varphi|) \cdot O(n)$ *on graph* G *with* n *vertices.*

While powerful, this is a meta theorem and hence, gives really crude bounds ($2^{2^{2^{\cdots}}}$ where height is the tree-width), so quite a lot of papers first put a problem in FPT using Courcelle's theorem and then find better bounds.

Also, tree-width allows us to get sub-exponential algorithm on planar graphs. By some meta theorem, we can show that there is $O^*\left(2^{O(\sqrt{k})}\right)$ algorithm for planar vertex cover. This was later lifted to genus G.

So why is this useful? There is a library LIGAND that is 9712 organic compounds that turn up a lot in chemistry. The tree width of anything on that database was ≤ 4 with only one molecule having it be 4. As nature, in some sense is simple and is not addversarially trying to screw our algorithms.

For a C programme, Thorup proved that the control flow graph (directed graph where the vertices correspond to the lines and there is an (directed) edge between 2 vertices (i, j) if there exists some invocation of the programme where $i \to j$).

Definition: Weighted Vertex Cover

Input: Undirected graph G, weights $w : V(G) \to \mathbb{N}$

Output: The least weight $W(S) = \sum_{s \in S} w(s)$ of a vertex cover S of G

We can solve this trees in polytime. Choose a root r and let $\text{dp}[v][p], \text{dp}[v][d]$ be the min weight vertex cover on the subtree rooted at v where v is **picked** in the vertex cover and v is **dropped** in the vertex cover respectively.

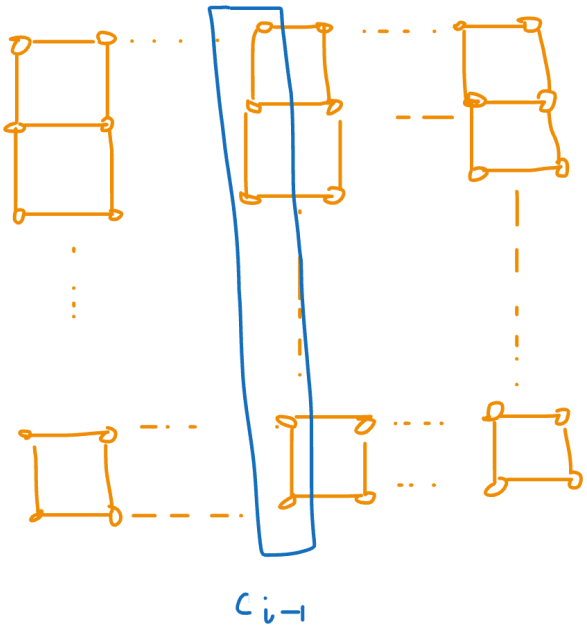
$$\text{dp}[v][p] = \sum_{c \in \text{ children of } v} (\min \text{dp}[c][p], \text{dp}[c][d])$$

$$\text{dp}[v][d] = \sum_{c \in \text{ children of } v} \text{dp}[c][p]$$

And we report the answer as $\min(\text{dp}[r][p], \text{dp}[r][d])$.

Let's now consider this problem on a grid $k \times N$ where $k < N$.

Can we solve this in $O^*(2^k)$ time?



For each subset S of C_i store: the least weight of a vertex cover of $G[C_1 \cup C_2 \cup ... \cup C_i]$ that intersects C_i exactly as S.

This immidietly gives us a $O^*(2^k \cdot 2^k) = O^*(4^k)$ algorithm.

Basically, if we have a small seperator, then we can use a DP style approach.

Definition: Path Decomposition and Path Width

A **path decomposition** of a graph G is a sequence $\langle B_1, B_2, ..., B_r \rangle$ of subtree of $V(G)$, each bag B_i such that:

1. Vertex coverage: every vertex is in some bag
2. Edge coverage: for each edge (u, v) there is a bag that contains both u and v.
3. Connectivity: For each vertex v, the set of bags that contain v appear consecutivly in the sequence.

The **width** of this path decomposition is $\max_i |B_i| - 1$.

The **pathwidth** of a graph G is the least width of a path decomposition of G.

Finding the pathwidth is NP Hard. Although, there are some polynomial time approximation algorithms.

Corollary 20.2. *The path width of a path is* 1.

Proof. Consider a path $v_1 - v_2 - ... - v_n$. Now, we can path decompose as $(v_1, v_2) - (v_2, v_3) - ... - (v_{n-1}, v_n)$.

This gives a path width of 1.

Note, we can't have lower as every edge needs to be in a bag so atleast one bag needs to have 2 vertices, that is size 2. ■

For grids, we can have a path decomposition of width $2k - 1$ by sliding window and k by slithering a snake across column + 1 vertice in the next column.

What is the algorithmic use? We can do a DP over the subsets of the bags (the exact intersection of the solution of everything before it).

In tree width, we will just do it with trees. Notice, trees can have massive path widths (consider a binary search tree).